PLAN ASSURANCE QUALITÉ

Table of Contents

Structure organisationnelle du projet (OBS) *4*

Clarté hiérarchique **4**

Cartographie des compétences **4**

Définition des rôles **5**

Mehdi BOUYDARNE : Développeur opérationnel 5

Paul DECAUCHY : Développeur backend 5

Pierre-Alexis THIEBOT : Développeur IA / ALGO 5

Grégoire VAILLANT : PO / Développeur full stack 6

NORMES *7*

Git **7**

Branch 7

Commit 7

Code **8**

GO 8

Pyhton 8

React 9

MÉTHODOLOGIE *10*

OUTILS *11*

Git / Github **11**

Jira **11**

OneDrive **11**

Google drive **11**

WhatsApp **11**

Figma **11**

TESTS *12*

DOCUMENTATION ONBOARDING *13*

Readme **13**

Contributing **13**

CHOIX TECHNOLOGIQUES *14*

Frontend – web **14**

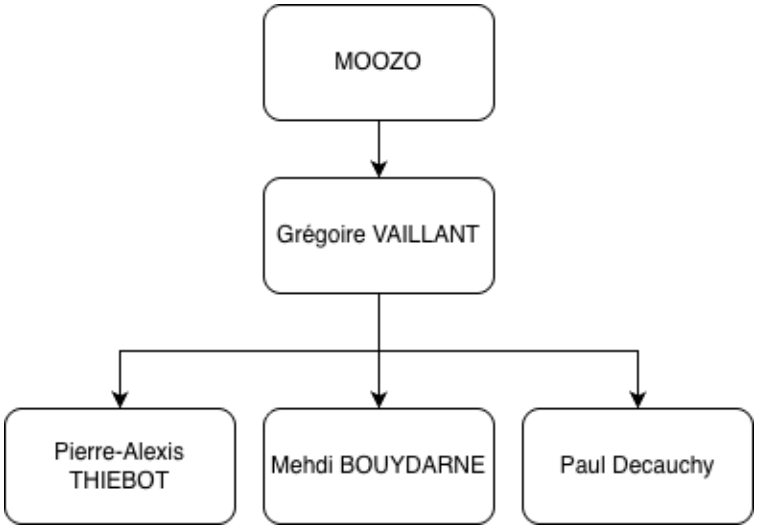
React.js 14

Frontend - mobile **15**

React Native 15
Backend (API) 15
Golang 15
Echo 16
Ogen 16
Backend (IA) 16
Python 16
PyTorch 16
Base de données 17
MongoDB 17
Déploiement 17
Docker 17

Structure organisationnelle du projet (OBS)

Clarté hiérarchique



L'équipe adopte une structure volontairement horizontale afin de favoriser la communication, l'autonomie et la réactivité.

Chaque membre est responsable de son périmètre technique tout en contribuant aux décisions collectives.

Cependant, afin d'éviter tout blocage décisionnel :

- Grégoire VAILLANT occupe le rôle de décisionnaire final.
- En cas de désaccord ou de nécessité de trancher rapidement, la décision finale est prise par Grégoire.
- Cette organisation permet de maintenir un équilibre entre collaboration ouverte et efficacité stratégique.

Cartographie des compétences

Pierre-Alexis				
Mehdi				
Paul				
Grégoire				

Définition des rôles

Mehdi BOUYDARNE : Développeur opérationnel

Après 3 ans de parcours généraliste en informatique, je me suis spécialisé en Cybersécurité et Cloud.

En alternance au sein de la cellule cyber d'une PME, je travaille sur l'automatisation et l'industrialisation des déploiements, la gestion d'infrastructure et la surveillance (mise en place d'une infrastructure de collecte/monitoring, construction d'un SIEM). Je contribue également à la sécurisation de l'infrastructure (durcissement, contrôle des accès, bonnes pratiques d'exploitation).

Paul DECAUCHY : Développeur backend

Développeur backend spécialisé en Go, j'évolue actuellement chez Ubisoft au sein du Lab, une entité dédiée à l'exploration de nouveaux usages technologiques.

Je conçois des systèmes backend distribués en appliquant les principes du Domain-Driven Design (DDD), une approche qui consiste à structurer l'architecture autour du métier afin de garantir cohérence et évolutivité.

Je travaille notamment sur l'implémentation de workflows asynchrones via Uber Cadence, ce qui implique la gestion de processus complexes et de longue durée, avec des mécanismes de reprise automatique en cas d'erreur et de maintien de la cohérence des données.

J'interviens également sur l'intégration des données avec les serveurs de jeu, en veillant à assurer performance, fiabilité et qualité logicielle.

Pierre-Alexis THIEBOT : Développeur IA / ALGO

Mon profil plutôt technique. Après 3ans en développement de jeux-vidéo où j'ai développé de solides compétences en algorithmique et en optimisation, je suis actuellement en spé cybersécurité, ce apporte une compréhension des architectures, des performances et des enjeux liés à la sécurité des systèmes.

Je travaille aussi dans une startup développant un SaaS de simulation de campagnes de phishing à des fins de sensibilisation, où je touche à tout sauf au front.

J'y ai notamment développé un **evil proxy** (proxy intermédiaire capable d'intercepter et de relayer des flux pour analyser des mécanismes d'authentification dans un cadre de test),

des systèmes d’envoi de campagnes de phishing simulées ainsi que des mécanismes de sécurisation. Cette expérience renforce mon intérêt pour les systèmes intelligents, l’automatisation et la compréhension fine des architectures techniques.

Même si je ne suis pas en spé IA, l’IA et la conception d’algorithmes sont des domaines qui me passionnent particulièrement.

Grégoire VAILLANT : PO / Développeur full stack

Mon profil est hybride, j’ai une formation initiale en école de commerce à emlyon business school, puis une spécialisation technique en développement informatique à EPITECH. Cette double formation me permet de comprendre à la fois les enjeux stratégiques d’un projet (vision produit, priorisation, cohérence globale) et ses contraintes techniques concrètes.

Au cours de mon alternance chez Quiz Room, j’ai été impliqué à la fois dans le développement full stack (MERN, API, AWS S3, gestion serveur) et dans des échanges organisationnels et fonctionnels. Cette position intermédiaire m’a naturellement amené à faire le lien entre les besoins métier et l’implémentation technique.

NORMES

Afin de garantir la qualité du code, la maintenabilité du projet et une collaboration efficace entre les membres de l'équipe, plusieurs normes de développement ont été mises en place. Ces normes portent principalement sur le contrôle de version, la structure des branches, les conventions de validation et les normes de codage pour chaque langage de programmation utilisé dans le projet.

Git

Le projet utilise Git pour la gestion des versions du code. Le dépôt est hébergé sur GitHub afin de faciliter la collaboration, la gestion des pull requests et les revues de code.

Chaque nouvelle fonctionnalité ou correction est développée dans une branche dédiée. Le développement ne se fait jamais directement sur la branche principale.

Branch

- main : version stable du projet
- develop : branche principale de développement
- feature/nom_feature : développement d'une nouvelle fonctionnalité
- fix/nom_fix : correction d'un bug

Une Pull Request doit être créée avant toute fusion dans develop. Celle-ci doit être validée par au moins un autre membre de l'équipe.

Commit

Les messages de commit suivent une convention simple inspirée du standard Conventional Commits afin de faciliter la lecture de l'historique du projet.

- feat: ajout d'une fonctionnalité
- fix: correction d'un bug
- refactor: modification interne du code sans changement fonctionnel
- docs: modification de la documentation
- test: ajout ou modification de tests

Exemple :

- feat: ajout de la gestion du plan de table
- fix: correction du calcul de position des invités

Code

Plusieurs règles communes sont appliquées afin d'assurer la lisibilité et la cohérence du code :

- Indentation : 4 espaces
- Longueur maximale d'une ligne : environ 100–120 caractères
- Nommage des variables : noms explicites et compréhensibles
- Fonctions : doivent rester courtes et avoir une responsabilité unique
- Commentaires : uniquement lorsque le code n'est pas immédiatement compréhensible
- Gestion des erreurs : chaque erreur doit être explicitement gérée et retournée correctement

Le formatage automatique du code JavaScript est réalisé avec Prettier afin de garantir une cohérence entre tous les développeurs.

GO

- PascalCase : structs, interfaces, func et const public
- CamelCase : var, func private, type de pointeur
- ID, URL, HTTP, API, SQL, DB : en majuscule dans les *var* ou *func*
- Imports en haut du fichier
- Plus d'infos

Python

- snake_case : variables, fonctions, méthodes
- PascalCase : classes, exceptions
- UPPER_CASE : constantes
- _leading_underscore : variables ou fonctions privées (usage interne au module)
- **dunder** : méthodes spéciales Python (__init__, __str__, etc.)
- ID, URL, HTTP, API, SQL, DB : en majuscules dans les noms de variables ou fonctions
- Imports en haut du fichier
- Ordre des imports :
- bibliothèques standard
- bibliothèques externes
- modules du projet
- Indentation : 4 espaces (pas de tabulations)
- Longueur maximale d'une ligne : 79 caractères

- Docstrings obligatoires pour les classes et fonctions publiques
- Une fonction doit faire une seule chose et rester courte
- Utiliser try/except avce le bon type d’erreur pour gérer les erreurs
- Plus d’infos : PEP8 (Python Style Guide)

React

- Organisation des fichiers : Organisation moléculaire

MÉTHODOLOGIE

Pour le développement du projet Moozo, l'équipe a choisi d'utiliser la méthode Scrum, issue des méthodes Agile software development. Cette approche est particulièrement adaptée aux projets logiciels nécessitant des évolutions régulières et des ajustements en fonction des retours des utilisateurs ou des nouvelles contraintes techniques.

Contrairement à la méthode Waterfall model, qui repose sur une progression linéaire et séquentielle des phases (analyse, conception, développement, test), Scrum permet une organisation plus flexible basée sur des cycles courts appelés sprints. Cette approche facilite l'adaptation aux changements et permet de livrer régulièrement des fonctionnalités opérationnelles.

Dans le cadre du projet Moozo, l'utilisation de Scrum permet notamment : d'avancer progressivement sur les différentes fonctionnalités (gestion des invités, plan de table, partage de photos, etc.) de tester régulièrement les fonctionnalités développées d'identifier rapidement les problèmes techniques ou fonctionnels

L'équipe prévoit d'organiser un sprint par semaine, avec une réunion de suivi hebdomadaire permettant de faire le point sur l'avancement du projet, les tâches réalisées et les difficultés rencontrées. Cette réunion permet également de planifier les objectifs du sprint suivant.

Plusieurs livrables et outils de suivi seront produits afin de structurer l'avancement du projet :

- Compte-rendu de réunion (CR) afin de garder une trace des décisions et des points discutés
- Backlog produit listant l'ensemble des fonctionnalités à développer
- Burndown chart, utilisé pour suivre l'avancement du sprint et visualiser la quantité de travail restante
- Documentation technique, décrivant l'architecture et les choix technologiques du projet

Ces éléments permettent de suivre l'avancement du projet de manière structurée et de garantir une bonne coordination entre les membres de l'équipe.

OUTILS

Git / Github

Utilisation de Git et GitHub pour le versionnement du code et l'organisation des différents projets. Le développement se fait sur une branche créée depuis la branche develop lorsque le ticket passe en In Progress. Une pull request est ensuite ouverte avec au minimum une review externe avant validation, et les tests unitaires doivent être exécutés et validés avant le merge.

Jira

Jira (software) est utilisé pour la gestion des tickets, des user stories et du suivi de l' avancement des tâches. Le workflow classique est : To Do → In Progress → Review → Done. Le passage en Done est conditionné par la validation du code et des tests. Le merge d'une pull request entraîne la fermeture automatique du ticket lorsque l'intégration est configurée.

OneDrive

Microsoft OneDrive sert au stockage et au partage des documents administratifs, cahiers des charges, rapports et fichiers bureautiques au format Word ou similaire. L'objectif est d'avoir un référentiel documentaire unique et accessible à l'équipe.

Google drive

Microsoft OneDrive sert uniquement au stockage des graphiques fait avec draw.io

WhatsApp

WhatsApp est utilisé pour la communication rapide entre les membres de l'équipe pour les questions opérationnelles, la coordination ou les informations urgentes.

Slack

Slack sera utilisé pour intégrer un bot chargé de publier automatiquement des informations liées au dépôt de code ainsi qu'aux environnements de développement et de production (notifications de commits, déploiements, incidents, etc.).

Figma

Figma est utilisé pour la conception des maquettes, prototypes d'interface et validation des écrans avant la phase de développement. Les designs y sont partagés pour recueillir les retours des parties prenantes.

TESTS

Unitaires / Intégration / End-to-End

Recette interne (dev) vs QA client

Tests non-fonctionnels (sécurité, performance)

Coverage minimum attendu

DOCUMENTATION ONBOARDING

Readme

Le fichier **Readme** présente l'objectif de l'application, son positionnement ainsi que les fonctionnalités principales du projet Moozo. Il décrit ce que l'on construit et comment le produit répond aux besoins des organisateurs, des invités et des prestataires. On y retrouve aussi les informations de démonstration, les recrutements en cours ainsi que les auteurs du projet.

Contributing

Le fichier **Contributing** explique les règles pour participer au développement du projet. Il détaille le workflow Git à respecter, les standards de code, le format des commits, le processus de pull request ainsi que les bonnes pratiques de qualité logicielle afin de garantir un développement cohérent et maintenable.

CHOIX TECHNOLOGIQUES

Frontend – web

React TS

Le frontend web du projet Moozo est destiné aux invités afin de leur permettre d'accéder facilement aux informations de l'événement sans installation d'application mobile. Le choix de React avec TypeScript répond à cet objectif d'accessibilité, de performance et de maintenabilité.

React est une bibliothèque JavaScript open-source maintenue par Meta, utilisée pour construire des interfaces utilisateurs interactives basées sur des composants réutilisables. Selon l'enquête Stack Overflow Developer Survey 2024, React fait partie des technologies web les plus utilisées par les développeurs, ce qui garantit un écosystème mature et une large disponibilité de ressources techniques.

L'architecture component-based de React facilite le développement de ces fonctionnalités. Par exemple, le plan de table, la galerie de photos ou la liste des invités peuvent être développés sous forme de composants indépendants, réutilisables et facilement maintenables.

React utilise également un mécanisme appelé Virtual DOM, qui optimise la mise à jour de l'interface lorsque les données changent. Cela améliore la fluidité de l'expérience utilisateur dans des interfaces dynamiques où les contenus peuvent être mis à jour en temps réel (par exemple lors du partage de photos ou de l'ajout d'informations sur l'événement).

L'utilisation de TypeScript, développé par Microsoft, permet d'ajouter un typage statique au code JavaScript. Cela améliore la fiabilité de l'application et réduit les erreurs lors de la manipulation de données structurées comme les invités, les tables ou les informations d'événement. Le typage facilite également la maintenance et la collaboration sur un projet de taille importante.

Ainsi, React avec TypeScript constitue un choix adapté pour le frontend web de Moozo en offrant une interface performante, maintenable et accessible aux invités depuis n'importe quel navigateur sans installation préalable.

https://survey.stackoverflow.co/2024/technology#2-web-frameworks-and-technologies

Frontend - mobile

React Native

L'application mobile du projet Moozo est destinée principalement aux organisateurs et aux prestataires, qui doivent gérer l'événement directement depuis leur smartphone. Pour cela, le framework React Native avec TypeScript a été choisi.

React Native est un framework open-source développé par Meta permettant de créer des applications mobiles pour Android et iOS à partir de la même base de code JavaScript. Contrairement aux solutions hybrides basées sur du HTML, React Native utilise des composants natifs, ce qui permet d'obtenir des performances proches des applications mobiles natives.

React Native permet de développer ces fonctionnalités tout en réutilisant les concepts et une partie de la logique utilisés dans React (composants, gestion d'état, hooks). Cette cohérence technologique réduit la complexité du développement et facilite la maintenance du projet.

Comme pour le frontend web, l'utilisation de TypeScript améliore la qualité du code grâce au typage statique. Cela est particulièrement utile dans une application manipulant de nombreuses entités (événements, invités, tables, photos, prestataires), car les erreurs peuvent être détectées dès la phase de développement.

React Native dispose également d'un écosystème riche de bibliothèques permettant d'intégrer facilement des fonctionnalités nécessaires dans une application événementielle, comme l'accès à la caméra, les notifications ou la gestion des médias.

Ainsi, React Native avec TypeScript permet de développer une application mobile performante et multiplateforme, adaptée aux besoins des organisateurs et prestataires du projet Moozo.

Backend (API)

Nos choix se font dans l'objectif d'avoir une api rapide et claire, un code lisible et maintenable

Golang

- Performance élevée
- Faible consommation de mémoire
- Ecosystème mature pour api REST

Pourquoi pas Node.JS/TS ? Lent, charge CPU, libs remplies de vulnérabilités

Echo

- Très performant (basé sur net/http)
- Middlewares simples à intégrer
- Grosse communauté

Pourquoi pas Gin ? Moins de choses natives, moins plug and play, trop verbeux

Ogen

- Génération de serveur complet
- OAPI First
- Validation automatique
- Boilerplate minime

Pourquoi pas oapi-codegen ? Plus manuel

Backend (IA)

Pourquoi l'intégrer dans un microservice séparé ?

- Le backend principal est en GO
- L'IA consomme beaucoup de CPU/RAM
- Les MàJ du service d'IA et de l'API ne sont pas synchrone
- Rollback/scalabilité indépendant

Python

- Ecosystème ML dominant
- Libs optimisées
- Enorme communautée

PyTorch

- Simple et intuitif
- Très utilisé en CV
- Debug facile
- Grosse communautée

Pourquoi pas TensorFlow ? Plus lourd et complexe, fait pour de gros projets avec beaucoup de stats

Base de données

MongoDB

- Orienté document
- Pas besoin de migrations lourdes
- Ajout de champ dynamique
- Sharing

Pourquoi pas SQL ? Scalabilité plus lourde, scalabilité horizontale

Déploiement

Docker

- Déploiement simple pour une infra simple
- Bonne intégration CI/CD
- Intuitif

Pourquoi pas k8s ? Surdimensionné pour le projet, complexe, choix pour plus tard